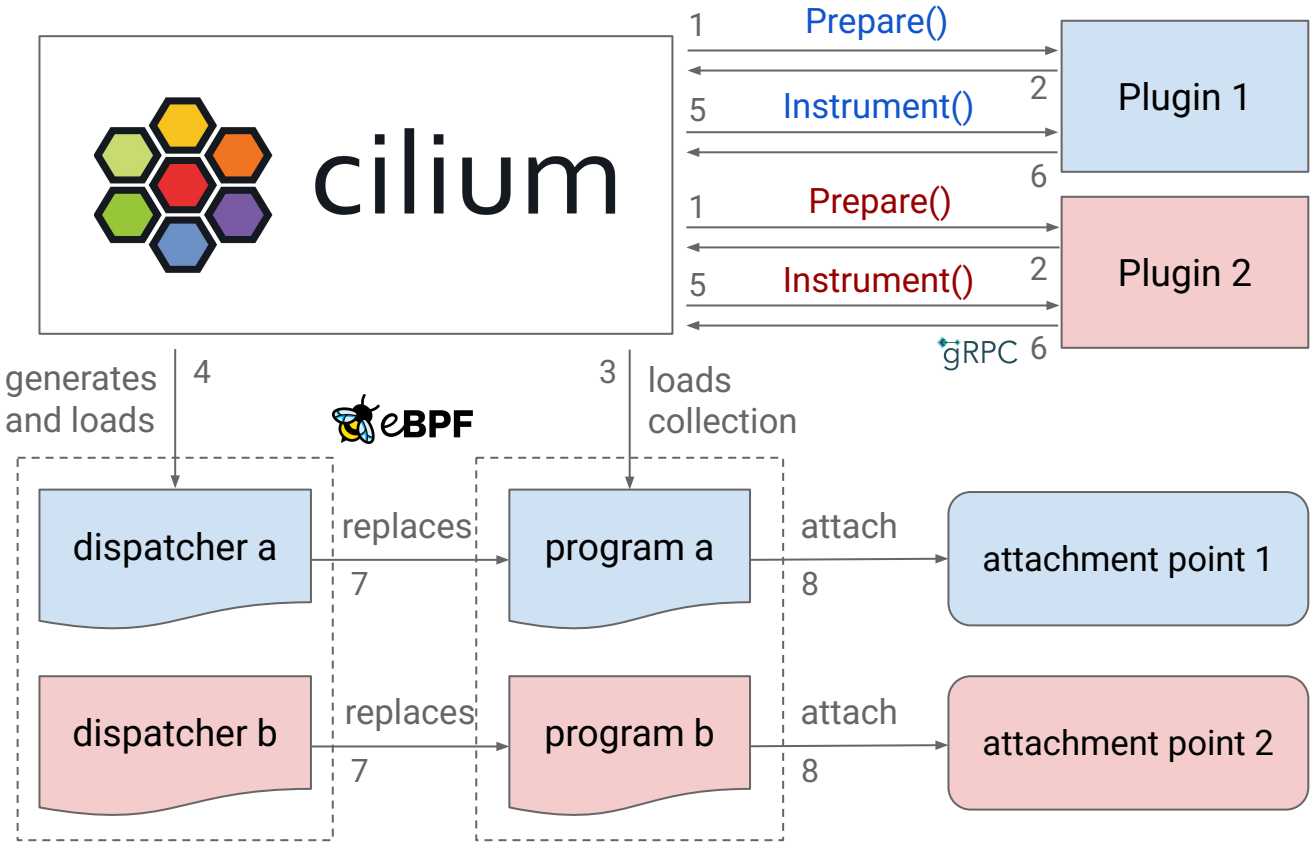
Cilium Dev Summit

bowei@, jrife@, tsotne@, arighnac@

Shared externally

Datapath Plugins

Datapath Plugins: Recap And Overview



Phase 1 (Prepare): Cilium loads a collection spec as it prepares for endpoint regeneration. Plugins tell Cilium which programs they want to instrument and provide ordering constraints relative to other plugins.

Phase 2 (Instrument): Cilium generates *dispatcher programs* that replace programs in the collection. Plugins load pre/post hook programs and Cilium inserts them in the appropriate place.

Datapath Plugins Updates

- CFP-41634: Datapath Plugins merged
- PR is up and needs reviews
  - Try it out! `make kind-image-datapath-plugin && make kind-install-cilium-datapath-plugin`

Datapath Plugins: Next Steps

- Better E2E coverage
  - Add e2e test coverage in CI that tests when a datapath plugin is installed.
- Tail call instrumentation
  - Not possible to use `freplace` with programs in PROG_ARRAY map, so explore alternatives
    - https://lore.kernel.org/all/20241015150207.70264-2-leon.hwang@linux.dev/
- Some other possible future enhancements:
  - Ability to override or instrument calls to BPF helpers or kfuncs.
  - Map replacements: plugin tells Cilium it wants to replace a map in the collection with its own.

Datapath Plugins And dpgen

- In the current PR, Cilium does not yet pass collection constants through to the plugin, but there's a placeholder for it in the API.
- jrife developed a PoC that does BPF -> `dpgen` -> protobuf IDL -> Go structs
  - You can't represent integers smaller than `int32` in protobuf and you need to use `[]byte` instead of `[n]byte`.
  - Because of this, you need to do size checks and cast down to smaller ints in `config.Map()` before populating a BPF collection with `Constants`. In my PoC I generated per-type Map functions which do the casting and invoked those from `config.Map()`
- This needs more discussion before deciding on a direction, so it’s out of scope for the initial PR.
- Goal is to have the same config that Cilium consumes in `LoadCollection` passed to plugins.

jordan@t14: ~/code/cilium

```
jordan@t14:~/code/cilium$ make kind-install-cilium-datapath-plugin
```

jordan@t14: ~

```
jordan@t14:~$ sudo cat /sys/kernel/debug/tracing/trace_pipe
[sudo] password for jordan:
```

Scaling

Scaling Cilium Network Policy

	Symptoms at Scale	Root Causes
Identity Allocator	Stalls, selector-cache thrash	kvstore latency, 16-bit ID limit, watcher backlog
Policy Repository	Long merge times, queue growth	Per-update trie rebuild, locks, no diffing
Endpoint Regeneration	Build queue spikes, stale revisions	Serial mutex, full recompiles
BPF Maps	Update failures, drops (POLICY_DROP)	Hard limits on Policy Map (16k be default), IPCache (512k)
L7 Proxy	Envoy CPU/RSS spikes, redirect fails	Single shared Envoy, costly config reloads

Identity Exhaustion and IPCache Capacity

Problem: Cilium uses a 16-bit bitmask for local identities.

Scale Impact: Max 65,536 unique security identities. 300k pods cannot scale without massive identity sharing and label normalization.

- pkg/identity/numericidentity.go

Problem: Global IPCache is hard-capped at 512k entries.

Scale Impact: 300k dual-stack pods = 600k IPs. This guarantees map overflow, update failures, and massive traffic drops.

- pkg/maps/ipcache/ipcache.go

Policy: The “Global Lock” && Endpoint Regeneration

Problem: Global RWMutex combined with O(Rules) resolution complexity.

ReplaceByResource and resolvePolicyLocked in pkg/policy/repository.go.

Scale Impact: 50k rules + 20k nodes = exponential lock contention. A single rule update can stall the entire control plane.

Problem: BPF compilation is gated by a global compilationLock and parallel worker semaphore.

Full re-builds triggered by configuration changes in pkg/endpoint/bpf.go.

Scale Impact: "Regeneration storms" spike CPU to 100% and delay policy enforcement by minutes during cluster rollouts.

L7 Proxy: The “Shared Context Bottleneck”

- **Problem**: Single shared Envoy instance per node for ALL pod L7 traffic.
- xDS config reload pushes in pkg/endpoint/bpf.go for every L7 rule update.
- **Scale Impact**: Policy churn scales linearly with pod density, but proxy reloads happen globally per node, leading to Envoy process starvation.

Arena Shared Policy Map Architecture?

- Per-endpoint policy maps duplicate common rules, wasting significant memory.
- Introduce a node-scoped shared arena map.
- Drastically reduce memory footprint on nodes with many pods.
- Evaluating rule deduplication strategies within the legacy architecture to address the constraints of 6.9+ kernel requirements and limited data structure adoption.

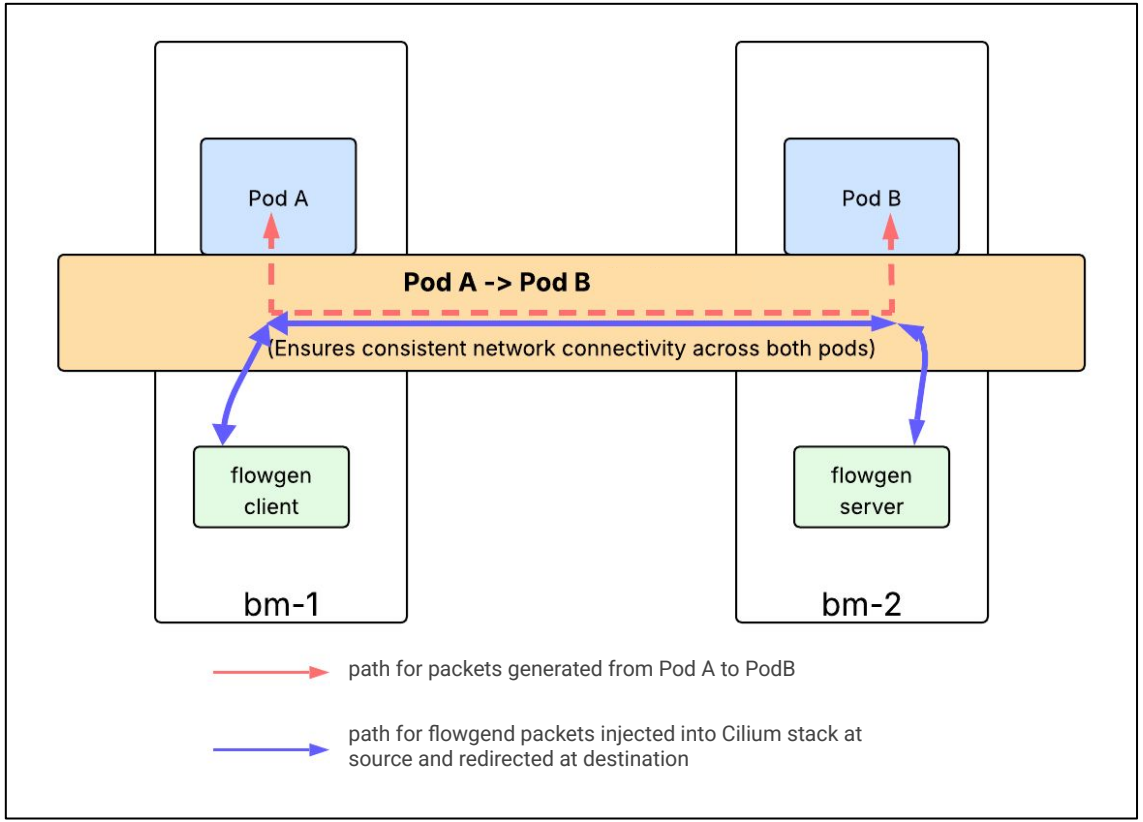
Tracing / Probing

Synthetic Probing: FlowGen

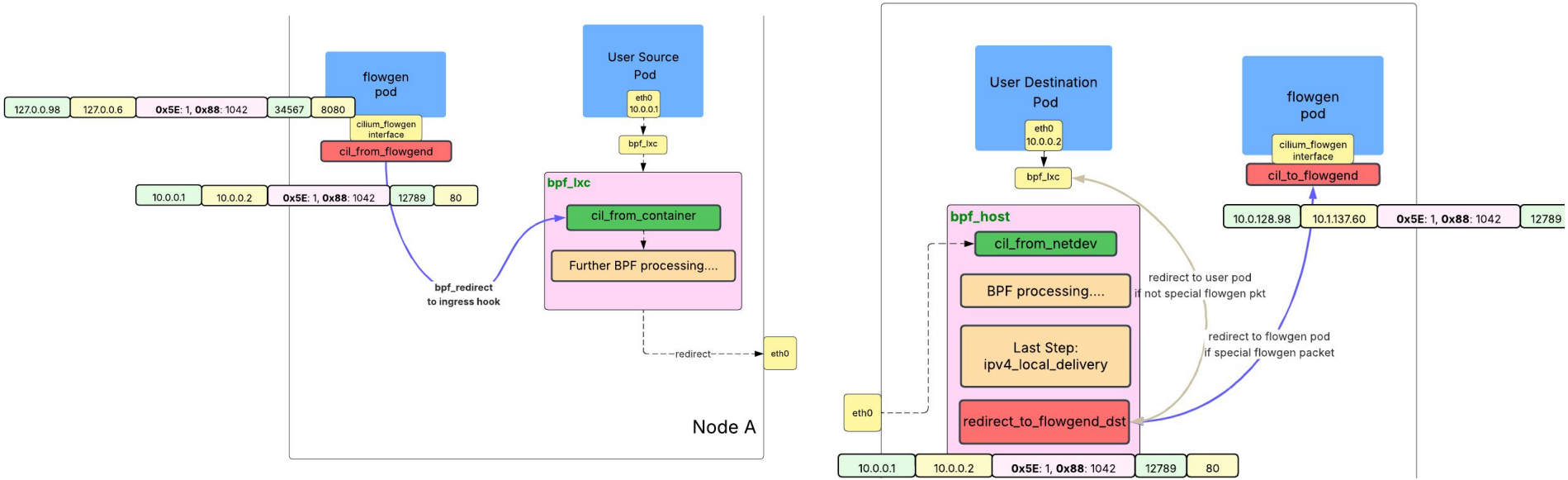
Generate **Synthetic Network Flows** (probes) across user-specified endpoints with the capability of generating distributed traces of these flows (using the distributed tracing architecture), providing deep visibility into the overlay.

These network probes look exactly like as if it originated from the user source endpoints. Tracing these "sentinel" flows gives a complete understanding about the functional behavior of the network stack for a specific flow.

Similar to: Antrea Traceflow



Synthetic Probing: FlowGen



Scalability tests

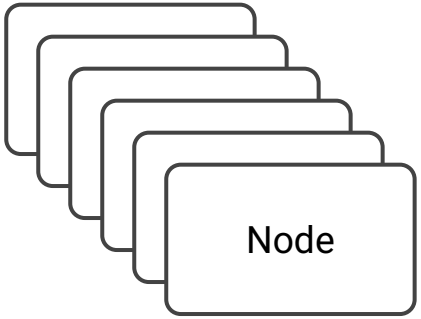
Scalability tests - problem

- Expensive
- Terrible feedback loop
- Flaky
- Hard to add/modify test cases

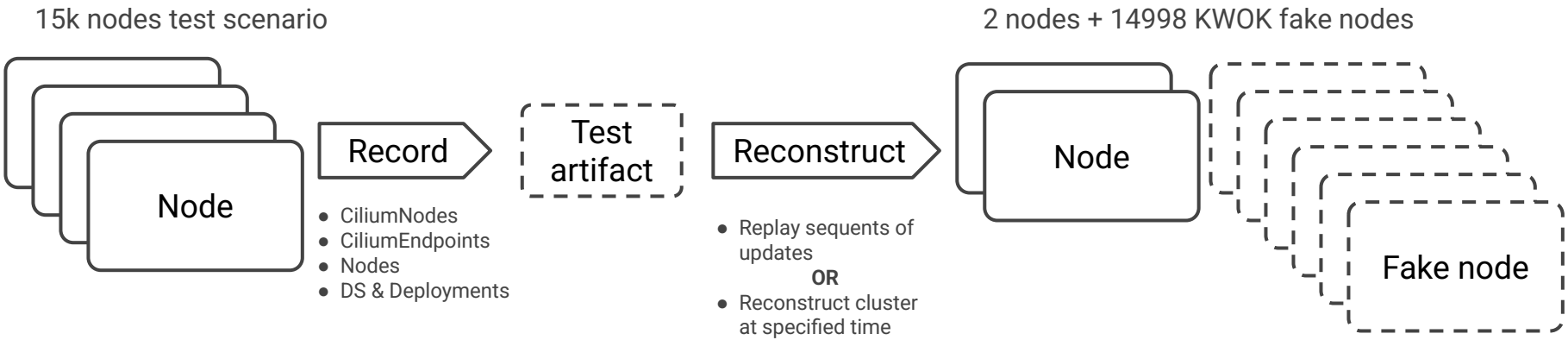
Working with scale tests right now:

1. Run test
2. Wait 12h
3. Test fails
4. Debug for 1h, startup SLO for agent not met
5. Add detailed logs
6. Run test with modified agent
7. Wait another 12h
8. Test fails after 2h due to infrastructure issues

15k nodes



Scalability tests - idea



- Test Cilium performance at scale without the scale.
- Replay sequence of updates to test operator/agent in isolation.

Scalability tests - goal

- Simpler testing, easier to create own test cases
- Run it locally or on smaller scale
- Faster feedback loops